

Documentation de la Base de Données

Ce document présente les choix techniques, conceptuels et l'architecture de la base de données relationnelle conçue pour l'application de gestion de mosaïques. La base de données gère les clients, les commandes, la composition des produits (mosaïques), ainsi que les stocks de pièces élémentaires.

1. Choix Techniques

1.1 Système de Gestion de Base de Données (SGBD)

- **SGBD** : MySQL / MariaDB.
- **Moteur de Stockage** : InnoDB.
 - *Justification* : InnoDB a été choisi pour sa gestion des transactions (ACID), essentielle pour garantir l'intégrité des commandes et des mouvements de stocks. Il supporte également les clés étrangères (FOREIGN KEY), assurant la cohérence référentielle des données.

1.2 Encodage

- **Jeu de caractères** : utf8mb4.
- **Interclassement** : utf8mb4_general_ci.
 - *Justification* : utf8mb4 est le standard moderne permettant de stocker l'ensemble des caractères Unicode (y compris les emojis et caractères spéciaux), contrairement à l'ancien utf8 de MySQL. Cela garantit une compatibilité internationale pour les données clients et les descriptions produits.

1.3 Conventions de Nommage

- **Tables** : PascalCase (ex: CustomerOrder, MosaicComposition). Cette convention permet de distinguer clairement les entités.
- **Colonnes** : Snake_case (ex: id_Customer, bank_name) pour une lisibilité accrue dans les requêtes SQL complexes.
- **Clés Primaires** : Préfixées par id_ suivi du nom de l'entité (ex: id_Item), sauf exceptions sémantiques.

2. Choix Conceptuels et Modélisation

2.1 Décomposition des Produits (Items)

Plutôt que d'avoir une table unique géante pour tous les types de pièces, nous avons opté pour une normalisation stricte (3NF) :

- La table Item représente une pièce générique.
- Les caractéristiques sont externalisées dans des tables de référence : Shapes (Formes) et Colors (Couleurs).
- *Avantage* : Évite la redondance de données textuelles (ex: répéter "Rouge" 1000 fois) et facilite la maintenance des attributs.

2.2 Gestion des Mosaïques (Composition)

La structure suit un modèle de "Nomenclature" (Bill of Materials) :

- **Mosaic** : L'objet fini vendu au client.
- **MosaicComposition** : Table d'association liant Mosaic à Item. Elle définit de quoi est faite une mosaïque (combien de pièces de quel type).
- *Contraintes* : ON DELETE CASCADE est appliqué sur la composition. Si une mosaïque est supprimée, sa recette (composition) l'est aussi, évitant les orphelins.

2.3 Gestion de l'Immuabilité (Commandes)

Un point critique dans la conception e-commerce est l'immuabilité des commandes passées.

- La table CustomerOrder enregistre l'entête de la commande.
- La table OrderItem lie la commande aux articles (Item).
- **Choix fort** : Bien que liée à Item, la table OrderItem doit conceptuellement "figer" l'état de la vente au moment T. Si le prix d'un Item change dans le futur, cela ne doit pas impacter les commandes passées.

2.4 Gestion des Stocks (Traçabilité)

L'existence de la table StockEntry indique un choix de gestion par "Journalisation" (ou *Event Sourcing* simplifié) plutôt que par simple compteur :

- Au lieu de modifier uniquement une colonne quantity dans Item, on peut enregistrer les mouvements dans StockEntry.
- *Avantage* : Permet une traçabilité totale (audit) de qui a ajouté/retiré du stock et quand.

3. Logique Métier (Procédures, Triggers et Vues)

Cette section décrit la logique implémentée (ou à implémenter) côté base de données pour garantir l'intégrité et faciliter l'accès aux données.

3.1 Triggers (Déclencheurs)

Trigger : TR_UpdateStock_AfterOrder

- **Type** : AFTER INSERT sur OrderItem.
- **Rôle** : Décrémenter automatiquement le stock disponible lorsqu'une ligne de commande est créée.
- **Logique** :
 1. Récupérer la quantité commandée.
 2. Mettre à jour la table Item (ou insérer une sortie dans StockEntry selon la stratégie choisie).
 3. Lever une erreur si le stock est insuffisant.

Trigger : TR_SecurePayment_Token

- **Type** : BEFORE INSERT sur BankDetails.
- **Rôle** : Sécurité basique.
- **Logique** : S'assurer que le numéro de carte complet n'est jamais stocké en clair (seuls les last_four et le payment_token sont conservés).

3.2 Procédures Stockées

Procédure : PROC_CreateMosaicFromTemplate

- **Paramètres** : p_clientId, p_templateId.
- **Rôle** : Créer une nouvelle entrée dans Mosaic et copier les lignes de MosaicComposition d'un modèle existant vers la nouvelle mosaïque personnalisée du client.
- **Intérêt** : Encapsule une logique complexe d'insertion multiple dans une transaction unique.

Procédure : PROC_AnnualRevenue

- **Paramètres** : p_year.
- **Rôle** : Calculer le chiffre d'affaires pour une année donnée en agrégeant les données de CustomerOrder et OrderItem.

3.3 Vues (Views)

Vue : V_StockAlert

- **Description** : Affiche uniquement les items dont le stock est critique (par exemple < 10 unités).
- **Usage** : Utilisée par le tableau de bord administrateur pour le réapprovisionnement.
CREATE VIEW V_StockAlert AS

Vue : V_ClientOrderHistory

- **Description** : Vue dénormalisée simplifiant l'affichage de l'historique client (jointure pré-faite entre Client, Commande et Mosaïque).

5. Sécurité et Droits

- **Utilisateur Applicatif** : L'application PHP/Java ne doit pas se connecter en root. Un utilisateur spécifique app_user doit être créé avec des droits restreints (SELECT, INSERT, UPDATE, DELETE) sur les tables de la base SAE_S3_BUT2_INFO uniquement.
- **Injection SQL** : L'utilisation de types stricts (int, date) dans la structure force une première validation des données, mais les requêtes préparées côté applicatif restent obligatoires.
-

6. MCD

